최기환

Frontend Engineer

반복되는 문제를 구조와 도구로 바꿉니다

rlghks3004@gmail.com GitHub gihwan-dev.tistory.com

DB 성능 모니터링 제품 MaxGauge의 웹 프론트엔드를 만듭니다. 같은 문제가 되풀이되면 공통 구조나 도구로 바꿔서, 다음 사람은 신경 쓰지 않아도 되게 하는 일을 가장 많이 해 왔습니다.

경력

주식회사엑셈 · 프론트엔드 개발자 2024.11 - 현재

MaxGauge 6 · MaxGauge 5 · 사내 테이블 엔진 · 사내 AI 도구

MaxGauge 6 · 차세대 DB 성능 분석 웹 제품 2025.02 - 현재

`React · TypeScript · TanStack Query · ECharts`

느린 SQL과 세션의 원인을 찾는 분석 화면을 만들고, 화면 공통 구조와 팀 개발 환경을 맡고 있습니다.

공통 구조

- **22개 화면 테이블 통일**
  - 화면마다 테이블을 따로 만들어 정렬·필터 동작과 컬럼 이름·단위가 제각각이었음
  - 범용 동작은 공통 엔진에, 컬럼 정의·상세 링크 같은 제품 규칙은 어댑터에 두고 화면은 컬럼과 데이터만 선언하게 나눔
  - 컬럼 259개를 레지스트리 하나에 모아 이름·타입은 화면에서 바꿀 수 없게 함
  - 22개 화면이 같은 컬럼 이름과 동작을 씀
  - 컬럼 스펙이 바뀌면 레지스트리 한 곳만 고치면 됨
- **분할 화면 레이아웃 규칙 통일**
  - 차트·테이블 분할 화면 5개가 레이아웃 복원 규칙을 각자 조립해, 한 곳만 바뀌면 동작이 어긋날 수 있었음
  - 조회 탭과 레이아웃 생명주기를 함께 만드는 함수 하나로 규칙을 모음
  - 5개 화면이 이 함수 하나만 쓰게 됨
- **소비처 없는 테이블 옵션 정리**
  - 패널 열림 방향, 컬럼 고정 방향처럼 스펙에 없는 옵션이 쓰는 곳 없이 공개 인터페이스에 남아 있었음
  - 화면을 옮기기 전에 스펙에 필요한 최소 인터페이스만 남기기로 함
  - 조건 필터 바의 공개 props를 1개로 줄이고 나머지는 테이블 상태에서 파생
- **불필요한 차트 리사이즈 최적화 제거**
  - 크기가 바뀌는 동안 캔버스를 늘려 보여 주고, 멈춘 뒤 200ms가 지나야 다시 그리는 최적화가 이미 들어가 있었음
  - 늘어난 캔버스에서 글자와 점이 찌그러졌고, 기획팀도 최적화 때문에 사용 경험이 나빠지는 것을 원하지 않았음
  - 가장 무거운 차트와 API가 줄 수 있는 최대 데이터로 반영 방식 10가지를 비교하는 실험 화면을 만들어 측정
  - 끊김은 21만 점 Scatter 같은 드문 경우에만 생겼고, 납품 전인 지금 이 경우를 위해 대부분의 화면 경험을 포기할 필요는 없다고 판단
  - 최적화를 걷어내고 크기 변경마다 프레임당 한 번 다시 그리게 함
  - 대용량 차트는 실제로 문제가 보고되면 성능 기준선을 잡고 따로 처리하기로 남김

성능

- **SQL 목록 이동 대기 단축**
  - 차트에서 SQL을 고르면 목록이 그 앞 페이지를 전부 받은 뒤에야 행으로 이동해, 뒤쪽 행이면 수 분을 기다려야 했음
  - 서버가 선택 행의 페이지를 알려 줄 수 있어, 그 페이지만 먼저 받고 앞뒤는 스크롤할 때 받기로 함
  - 공통 엔진에 아직 받지 않은 구간의 높이를 비워 두는 가상 공간과 행 이동 API를 추가
  - 실데이터 사례에서 목록 요청 178건 → 2건, 표시까지 212.5초 → 11.8초[1]
  - SQL Scatter와 SQL 상세 두 화면에 같은 방식을 적용

AI 개발 환경

- **화면 규약 스킬**
  - 분석 화면 규약이 코드·주석·개인 메모에 흩어져 있어 같은 위반이 반복해서 들어왔음
  - 규약 정본을 문서로 쓰고, 변경분을 그 규칙으로 검사해 고치는 스킬을 붙임
  - 누가 작업해도 결과물이 같은 규약을 따름
  - 스킬을 넣은 뒤로 해당 규약에 대한 리뷰 지적이 없어짐

- **Before/After 영상 녹화 스킬**
  - 스크롤·드래그처럼 움직임이 핵심인 변경은 리뷰어가 코드를 자세히 읽거나 직접 재현해야 판단할 수 있었음
  - 시나리오 파일 하나로 dev와 현재 코드를 같은 조작으로 녹화하고 비교 영상까지 만드는 스킬을 MR 생성 흐름에 연결
  - 리뷰어가 코드를 세세히 읽지 않고 영상으로 동작 차이를 빠르게 확인함

- **로컬 환경 구성 스킬**
  - 백엔드 서버 6종과 Oracle, 수집 데몬까지 있어야 화면이 돌아 로컬 셋업이 어려웠음
  - 공유 테스트 서버는 다른 본부와 함께 써서 마음대로 패치할 수 없었음
  - 실행 위치(노트북·사내 Linux)와 구성 절차를 나눠, 위치만 바꾸면 같은 명령으로 띄우게 함
  - 원하는 백엔드 빌드로 작업 환경을 빠르게 만들고 실제 환경에서 검증할 수 있게 됨

CI와 관측

- **CI 파이프라인 시간 단축**
  - 유닛 테스트 파일이 5개월 만에 27개에서 341개로 늘어 MR 파이프라인이 무거워졌음
  - 러너 부족을 의심했지만 25회를 재 보니 큐 대기 중앙값이 1.6초여서, 러너가 아니라 잡 실행 방식을 고침
  - 요청 때마다 컴파일하던 Storybook을 미리 빌드해 정적으로 서빙
  - jsdom 생성 비용이 테스트 로직보다 커서 순수 로직 테스트는 node 환경으로 분리
  - MR 파이프라인 중앙값 198초 → 94초[2]

- **Sentry 브라우저 프로파일링**
  - Sentry Profiles가 비어 있어 화면이 느린 원인을 함수 단위로 볼 수 없었음
  - 브라우저 프로파일링 통합과, 이를 허용하는 문서 응답 헤더를 추가
  - 화면 이동마다 트랜잭션별 프로파일과 플레임그래프를 받게 됨

- **디자인 시스템 오버라이드 린트**
  - 디자인 시스템 스타일을 덮어쓴 코드가 약 2,540건이었고, 83%가 6월 이후에 들어왔음
  - 디자인 확정 전이라 막지 않고, 커밋과 MR에서 새로 늘어난 위반만 보이게 함
  - 의도한 예외는 주석으로 남아 한 번에 모아 볼 수 있음

[1] 테스트 서버 실데이터 20,212건에서 같은 점을 골라 전후를 한 번씩 녹화한 기록입니다. [2] 당시 MR의 비교 기록입니다. 변경 전 20회, 후 4회의 중앙값이며 재시도와 캐시 재생성 회차는 뺐습니다.

exem-table · 사내 공통 React 테이블 엔진 2026.03 - 현재

`React · TypeScript · TanStack Table · Vitest · Playwright`

여러 제품이 함께 쓰는 테이블 엔진의 유지보수를 맡고 있습니다.

- **TanStack Table v9 전환**
  - 값 하나만 바뀌어도 테이블 전체가 다시 그려져, 10,000행에서 검색어를 칠 때 화면이 2초 가까이 멈췄음
  - 행·셀이 필요한 상태만 구독하게 하고, 셀 병합처럼 쓰지 않는 기능은 계산 경로를 건너뛰게 함
  - 10,000행에서 검색어 14자 입력 렌더 1,927ms → 61ms (같은 브라우저·데이터로 v8과 v9 비교, 5회 중앙값)
  - 30,000행 JS 힙 102.7MB → 27.1MB (GC 후 7회 중앙값)
  - 대신 넓어진 제네릭 때문에 타입체크 시간이 0.9초 → 2.1초로 늘었음
- **상태 소유 규칙 정리**
  - 값과 콜백 조합에 따라 "읽기 전용" 같은 중간 상태가 생겨 사용하는 쪽 의도와 다르게 동작했음
  - 상태 소유자를 제어·비제어 둘 중 하나로만 정함
  - 중간 상태가 없어져 상태를 누가 바꾸는지 한눈에 판단할 수 있게 됨
- **엔진 접근 경계**
  - 제품 파일 142개가 TanStack을 직접 불러, 엔진을 바꾸면 수정이 전체로 번졌음
  - 엔진 접근을 파일 하나로 모음
  - 페이지 조회·캐시처럼 화면마다 다른 일은 엔진에 넣지 않고 쓰는 쪽에 둠
  - 엔진 직접 import 142개 파일 → 1개
- **동작 중심 테스트**
  - 설정값과 mock 반환값을 되읽는 테스트가 많았고, 공개 옵션이 실제 화면 동작으로 이어지는지는 검증하지 못했음
  - 실제 브라우저에서 테이블을 여는 동작 검증으로 바꿈
  - 버그는 실패하는 테스트로 먼저 재현한 뒤 고침
  - 보강한 테스트로 구현 결함 6건을 찾아 실패 테스트로 남김

MaxGauge 5 · 이전 세대 제품군 (Oracle · Tibero · SQL Server) 2025.01 - 현재

`JavaScript · ExtJS · Vue · Playwright · Claude Code · Codex`

금융·공공·제조 고객사 20곳의 요청과 배포 버전을 맡고 있습니다. 2025년 하반기에는 6명 넘게 나눠 처리하던 수정을 2026년 7월부터 혼자 처리하며, 그 흐름을 AI 유지보수 하네스로 만들었습니다.

AI 유지보수 하네스

- **반복 결함을 리뷰 스킬로**
  - 레거시 수정에서 null 처리, 늦게 도착한 응답, Oracle/PostgreSQL 쌍 누락 같은 결함과 사이드 이펙트가 반복됐음
  - 배포 브랜치의 커밋 315건과 리뷰 코멘트 881건을 전수 조사해, 자주 나온 문제를 8개 리뷰 관점으로 정리
  - 관점마다 독립된 서브에이전트가 리뷰하고, 검증 에이전트가 확정한 결함만 고친 뒤 결함이 0건이 될 때까지 다시 리뷰
  - 리뷰 기록이 없으면 MR 생성과 배포 패키징이 멈추게 함
- **이미지만으로 리뷰하는 MR**
  - 레거시 수정은 리뷰어가 코드를 읽거나 직접 화면을 띄워야 판단할 수 있었음
  - 정상·경계·회귀 경로를 시나리오로 나누고, 시나리오마다 같은 조건의 수정 전·후 화면이 있어야 MR 본문이 만들어지게 함
  - AI가 구현·리뷰한 변경을 코드를 다시 읽거나 직접 띄우지 않고 시나리오 화면만 확인해 병합함
  - SQL 튜닝 제품 ExTuner 연동처럼 MR 54건으로 나뉜 신규 기능도 같은 방식으로 병합
- **의도 중심 수정과 회귀 테스트**
  - MR 본문에는 바뀐 동작과 수정 의도만 쓰고, 처리 규칙은 Logic map, 화면과 API 대응은 Spec map 문서로 남김
  - 모든 MR에 수정 전 실패·수정 후 통과하는 회귀 E2E 기록이 있어야 MR이 만들어지게 해 회귀를 테스트로 막음

검증 환경

- **mock DB 회귀 E2E**
  - 화면 하나를 확인하려 해도 Java 서버까지 띄워야 했음
  - 실제 DB 스키마를 본뜬 mock DB로 Java 서버 없이 브라우저 E2E를 돌리는 하네스를 만듦
  - 최근 2년간 병합된 MR 250건을 모두 회귀 테스트로 덮음
- **명령 하나로 로컬 풀스택**
  - 라이선스와 인스턴스를 DB에 직접 넣던 기존 셋업은 서버 버전이 조금만 달라도 깨졌음
  - 이 단계를 제품 UI 절차로 바꿈
  - `pnpm run dev` 하나로 고객사 배포본 기준 8개 버전 조합을 띄우고 8개 시나리오를 모두 통과

사내 AI 도구 2026.03 - 2026.06

Python · TypeScript · LLM

- **사내 검색 exemos**
  - 이슈·MR·문서·매뉴얼을 한 번에 찾는 검색 API에서 검색 경로와 운영 배포를 맡음
  - QA팀이 고객 지원에 쓰는 등 사내에서 널리 씀
- **GitLab 슬래시 커맨드**
  - AI를 쓰려면 개인 PC로 맥락을 옮겨야 했고, 결과를 팀이 함께 보기 어려웠음
  - MR·이슈 스레드 댓글에서 바로 AI를 부르는 봇을 만듦
  - 운영 오류는 Sentry로, 추론 과정은 Langfuse로 나눠 추적
  - 스레드 안에서 AI 결과를 함께 검증하고 리뷰하게 됨
- **Design Studio**
  - 기획자가 실제 제품 코드 위에서 화면을 만들고 명세를 쓰는 워크스페이스를 만듦
  - 지금도 기획팀이 프로토타이핑에 씀

학력

동의대학교 정보통신공학과 · 응용 소프트웨어 공학 부전공 | 2017.03 - 2024.03

최근 글 GIHWAN-DEV.TISTORY.COM · 개발 분류 최신 5개 · 2026.10.18 확인

지금 테스트 코드가 더더욱 중요한 이유
2026-09-06

Tanstack Table V9 마이그레이션 회고
2026-08-26

개발자로 살아남기 - 성장에 대한 고민
2026-08-09

리액트로 바라보는 정책과 메커니즘의 분리
2026-07-19

Zustand 스토어를 지역화하기
2026-07-19